OASIS VM 컨테이너 런타임 분석

개요

OASIS VM의 Podman 버전은 기밀 컴퓨팅 환경에서 컨테이너 기반 애플리케이션을 안전하게 실행할 수 있는 런타임 환경을 제공합니다. Podman과 podman-compose를 통해 컨테이너 오케스트레이션을 지원하며, TDX 환경에 최적화된 보안 기능을 제공합니다.

---

1. 컨테이너 런타임 아키텍처

전체 구조

```
Stage 2 (Podman) → Podman Runtime → Container Applications
                  ↓
                  podman-compose → Multi-container Applications
```

핵심 구성 요소

- **Podman**: OCI 호환 컨테이너 런타임
- **podman-compose**: Docker Compose 호환 오케스트레이션
- **cgroups v2**: 리소스 관리 및 격리
- **네임스페이스**: 프로세스, 네트워크, 파일시스템 격리

---

2. Podman 런타임 설정

패키지 구성 (stage2-podman.bb)

```
RDEPENDS:${PN} = " \
stage2-base \ # 기본 Stage 2 환경
podman \ # 컨테이너 런타임
podman-compose \ # 컨테이너 오케스트레이션
python3-unixadmin \ # 시스템 관리 도구
python3-json \ # JSON 처리
python3-logging \ # 로깅 지원
"
```

초기화 스크립트 (80-podman)

```
#!/bin/sh
set -e

# cgroups v2 마운트 (리소스 제한)
mount -t cgroup2 none /sys/fs/cgroup

# 공유 메모리 설정 (Podman 락)
mkdir -p /dev/shm
mount -t tmpfs none /dev/shm
```

```
# 컨테이너 디렉토리 정리
rm -rf /storage/containers/run
rm -rf /storage/containers/net
rm -rf /var/lib/cni

# 스토리지 디렉토리 생성
mkdir -p /storage/containers/run
mkdir -p /storage/containers/graph
mkdir -p /storage/containers/graph/tmp
mkdir -p /storage/containers/net

# TUN 디바이스 권한 설정
chmod 0666 /dev/net/tun

# Podman 시스템 마이그레이션
podman system migrate
podman system prune --external

# 컨테이너 시작
if [ -d "/etc/oasis/containers" ]; then
cd /etc/oasis/containers
podman-compose up --detach --remove-orphans --force-recreate &
```

```
fi
```

---

3. 컨테이너 스토리지 관리

스토리지 구조

```
/storage/containers/
├── run/ # 런타임 데이터
├── graph/ # 이미지 및 컨테이너 데이터
| └── tmp/ # 임시 파일
└── net/ # 네트워크 설정
```

스토리지 백엔드

- **overlay**: 기본 스토리지 드라이버
- **임시 스토리지**: 재부팅 시 데이터 소실
- **암호화**: LUKS2 + dm-integrity (디스크 모드)

볼륨 관리

```
# 바인드 마운트를 통한 데이터 공유
/var/lib/containers -> /storage/containers/graph
/run/containers -> /storage/containers/run
```

4. 네트워크 구성

네트워크 네임스페이스

- **격리**: 각 컨테이너는 독립적인 네트워크 네임스페이스
- **브리지**: 기본 podman 브리지 네트워크
- **호스트**: 필요시 호스트 네트워크 공유

CNI (Container Network Interface)

```
# CNI 설정 정리
rm -rf /var/lib/cni

# 네트워크 설정 디렉토리
mkdir -p /storage/containers/net
```

TUN/TAP 지원

```
# TUN 디바이스 권한 설정
chmod 0666 /dev/net/tun
```

---

5. 리소스 관리 (cgroups v2)

cgroups 마운트

```
mount -t cgroup2 none /sys/fs/cgroup
```

리소스 제한 기능

- **CPU**: CPU 사용량 제한
- **메모리**: 메모리 사용량 제한
- **I/O**: 디스크 I/O 제한
- **네트워크**: 네트워크 대역폭 제한

예제 리소스 제한

```
# podman-compose.yml
version: '3'
services:
app:
image: myapp:latest
deploy:
resources:
limits:
cpus: '0.5'
memory: 512M
reservations:
cpus: '0.25'
```

```
memory: 256M
```

---

6. 컨테이너 오케스트레이션

podman-compose 기능

- **Docker Compose 호환**: 기존 compose 파일 사용 가능
- **멀티 컨테이너**: 여러 컨테이너 조합 관리
- **의존성 관리**: 컨테이너 간 시작 순서 제어
- **볼륨 관리**: 데이터 볼륨 공유

컨테이너 정의 위치

```
/etc/oasis/containers/
├── docker-compose.yml # 또는 podman-compose.yml
├── .env # 환경 변수
└── volumes/ # 볼륨 데이터
```

자동 시작 프로세스

```
if [ -d "/etc/oasis/containers" ];  then
cd  /etc/oasis/containers
podman-compose  up  --detach  --remove-orphans  --force-recreate &
```

```
fi
```

---

7. 보안 기능

Rootless 컨테이너

- **비특권 실행**: 루트 권한 없이 컨테이너 실행
- **사용자 네임스페이스**: UID/GID 매핑을 통한 격리
- **제한된 권한**: 시스템 리소스 접근 제한

네임스페이스 격리

```
# 지원되는 네임스페이스
- PID: 프로세스 ID 격리
- NET: 네트워크 격리
- MNT: 마운트 포인트 격리
- UTS: 호스트명 격리
- IPC: 프로세스 간 통신 격리
- USER: 사용자 ID 격리
```

SELinux 지원 (선택적)

```
# SELinux 컨텍스트 설정
podman run --security-opt label=type:container_t myapp
```

8. 이미지 관리

이미지 저장소

```
# 로컬 이미지 저장
/storage/containers/graph/overlay-images/

# 이미지 레이어 관리
/storage/containers/graph/overlay-layers/
```

이미지 풀 및 관리

```
# 이미지 다운로드
podman pull docker.io/library/nginx:latest

# 이미지 목록
podman images

# 이미지 정리
podman image prune
```

이미지 보안

- **서명 검증**: 이미지 서명 확인 (선택적)
- **취약점 스캔**: 이미지 보안 스캔 (외부 도구)
- **신뢰할 수 있는 레지스트리**: 허용된 레지스트리만 사용

---

9. 런타임 모니터링

컨테이너 상태 모니터링

```
# 실행 중인 컨테이너
podman ps

# 컨테이너 로그
podman logs <container_id>

# 컨테이너 리소스 사용량
podman stats
```

시스템 이벤트

```
# Podman 이벤트 스트림
podman events

# 시스템 정보
```

```
podman system info
```

로그 관리

```
# 컨테이너 로그 위치
/storage/containers/graph/overlay-containers/<id>/userdata/ctr.log

# 시스템 로그
journalctl -u podman
```

---

10. 성능 최적화

스토리지 최적화

```
# 오버레이 스토리지 드라이버 최적화
- 레이어 캐싱 활용
- 불필요한 이미지 정리
- 스토리지 공간 모니터링
```

네트워크 최적화

```
# 네트워크 성능 튜닝
- 브리지 네트워크 최적화
```

```
- DNS 캐싱 설정
- 네트워크 정책 최적화
```

메모리 최적화

```
# 메모리 사용량 최적화
- 컨테이너 메모리 제한 설정
- 공유 라이브러리 활용
- 불필요한 프로세스 제거
```

---

11. 개발 및 디버깅

개발 워크플로우

```
# 1. 컨테이너 이미지 빌드
podman build -t myapp:latest .

# 2. 로컬 테스트
podman run --rm -it myapp:latest

# 3. compose 파일 작성
# /etc/oasis/containers/docker-compose.yml
```

```
# 4. 배포 테스트
podman-compose up
```

디버깅 도구

```
# 컨테이너 내부 접근
podman exec -it <container_id> /bin/sh

# 네트워크 디버깅
podman network ls
podman network inspect <network_name>

# 볼륨 디버깅
podman volume ls
podman volume inspect <volume_name>
```

로그 분석

```
# 상세 로그 출력
podman logs --follow --timestamps <container_id>

# 시스템 로그 확인
```

```
dmesg | grep  -i  podman

journalctl  -xe
```

---

12. 실제 사용 예제

간단한 웹 애플리케이션

```yaml
# /etc/oasis/containers/docker-compose.yml

version: '3'

services:

web:

image: nginx:alpine

ports:

- "80:80"

volumes:

- ./html:/usr/share/nginx/html:ro

restart: unless-stopped

deploy:

resources:

limits:

memory: 128M

cpus: '0.5'
```

마이크로서비스 구성

```yaml
version: '3'
services:
frontend:
image: myapp/frontend:latest
ports:
- "3000:3000"
depends_on:
- backend
environment:
- BACKEND_URL=http://backend:8080
backend:
image: myapp/backend:latest
ports:
- "8080:8080"
depends_on:
- database
environment:
- DB_HOST=database
- DB_PORT=5432
database:
image: postgres:13-alpine
```

```
environment:
- POSTGRES_DB=myapp
- POSTGRES_USER=user
- POSTGRES_PASSWORD=password
volumes:
- db_data:/var/lib/postgresql/data


volumes:
db_data:
```

배치 작업 컨테이너

```
version: '3'
services:
worker:
image: myapp/worker:latest
restart: "no"
volumes:
- /storage/data:/app/data
environment:
- WORKER_MODE=batch
- LOG_LEVEL=info
```

13. 제한사항 및 고려사항

현재 제한사항

- **영구 스토리지**: 재부팅 시 컨테이너 데이터 소실
- **네트워크**: 기본적인 네트워크 기능만 제공
- **GPU 지원**: GPU 가속 미지원
- **스웜 모드**: Docker Swarm 기능 미지원

성능 고려사항

- **I/O 오버헤드**: dm-verity 검증으로 인한 성능 영향
- **메모리 사용량**: 컨테이너 런타임 메모리 오버헤드
- **네트워크 지연**: 네트워크 네임스페이스로 인한 지연

보안 고려사항

- **이미지 신뢰성**: 신뢰할 수 있는 이미지 사용 필요
- **네트워크 보안**: 컨테이너 간 네트워크 보안 설정
- **리소스 제한**: 적절한 리소스 제한 설정 필요

---

14. 모범 사례

컨테이너 설계

```
# 1. 최소한의 베이스 이미지 사용
FROM alpine:latest
```

```
# 2. 비특권 사용자 실행
RUN adduser -D -s /bin/sh appuser
USER appuser

# 3. 단일 프로세스 원칙
CMD ["./myapp"]
```

보안 설정

```
# compose 파일에서 보안 설정
services:
app:
image: myapp:latest
read_only: true
tmpfs:
- /tmp
cap_drop:
- ALL
cap_add:
- NET_BIND_SERVICE
```

리소스 관리

```
# 적절한 리소스 제한 설정
deploy:
resources:
limits:
cpus: '1.0'
memory: 512M
reservations:
cpus: '0.5'
memory: 256M
```

이 컨테이너 런타임 환경은 기밀 컴퓨팅 환경에서 안전하고 효율적인 컨테이너 기반 애플리케이션 실행을 가능하게 하며, Podman의 보안 기능과 OASIS VM의 보안 아키텍처가 결합되어 높은 수준의 격리와 보안을 제공합니다.